

Практична робота №1. Множинна лінійна регресія методом градієнтного спуску

Предмет: Машинне навчання

Рівень: 4 рік навчання

Датасет: [Student Performance Dataset] [Link](#)

Мета: Реалізувати алгоритм множинної лінійної регресії з нуля за допомогою методу градієнтного спуску. Провести повний аналіз якості моделі, включаючи перевірку всіх шести класичних припущень МНК.

1. Постановка задачі

Ми маємо дані про успішність студентів. Наша ціль — передбачити кількісну оцінку успішності (Performance Index) на основі академічних факторів. Оскільки цільова змінна Performance Index є неперервною, використовуватимемо множинну лінійну регресію.

Модель: $y_i = \beta_0 + \beta_1 * x_{i1} + \beta_2 * x_{i2} + \dots + \beta_m * x_{im} + \varepsilon_i$

Завдання: Знайти вектор параметрів , який мінімізує функцію втрат — **середню Квадратичну Помилку (MSE)**.

Метод: Градієнтний спуск (оскільки аналітичне рішення $(X^T X)^{-1} X^T y$ неефективне для дуже великих датасетів).

2. Покрокова інструкція

1. Підготовка даних (Data Preprocessing)

1. Імпорт бібліотек: pandas, numpy, matplotlib.pyplot, seaborn, sklearn.model_selection (для train_test_split), sklearn.preprocessing (для OneHotEncoder, StandardScaler).

2. Завантаження даних: Завантажте файл Student_Performance.csv .

3. Розвідковий аналіз (EDA):

- * df.info(), df.describe().

- * Перевірка на пропущені значення (df.isnull().sum()).

- * sns.heatmap(df.corr(), annot=True) — побудуйте теплову карту кореляцій для чисельних змінних. Проаналізуйте кореляцію предикторів із цільовою змінною Performance Index.

4. Обробка категоріальних змінних:

* Для бінарних змінних (на кшталт Extracurricular Activities) можна використати LabelEncoder або просто зіставити значення (напр., {'No': 0, 'Yes': 1}).

5. Вибір ознак та цільової змінної:

Цільова змінна (target) — Performance Index.

* Ознаки (features) — всі інші релевантні стовпці, крім можливо тих, що є прямим наслідком Performance Index. Мета завдання — побудувати модель на основі академічних факторів.

6. Поділ на навчальну та тестову вибірки: `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`

7. Нормалізація/Стандартизація: Обов'язковий крок для градієнтного спуску.

Використайте StandardScaler для масштабування ознак (з `X_train` навчіть скалер, потім трансформуйте і `X_train`, і `X_test`). Не забудьте додати стовпчик одиниць для intercept β_0 після масштабування.

```
# Після розбиття на train/test
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Додаємо стовпчик одиниць для intercept
X_train_final = np.c_[np.ones(X_train_scaled.shape[0]), X_train_scaled]
X_test_final = np.c_[np.ones(X_test_scaled.shape[0]), X_test_scaled]
```

3. Реалізація методу градієнтного спуску з нуля

1. Ініціалізація параметрів: Ініціалізуйте вектор ваг β (наприклад, нулями або малими випадковими числами). Розмірність: $(m+1) \times 1$, де m — кількість ознак.

```
b = np.zeros(X_train_final.shape[1])
```

2. Гіперпараметри: Визначте кількість ітерацій (`epochs`) та швидкість навчання (`learning_rate`). Почніть, наприклад, з `epochs=1000`, `learning_rate=0.01`.

3. Функція для обчислення функції втрат (MSE):

```
def compute_cost(X, y, b):
    predictions = X.dot(b)
```

```

errors = predictions - y
mse = (1 / len(y)) * np.sum(errors ** 2)
return mse

```

4. Функція градієнтного спуску:

```

def gradient_descent(X, y, b, learning_rate, epochs):
    cost_history = [] # Для відстеження змін MSE
    m = len(y)

    for _ in range(epochs):
        # Обчислення передбачень
        .....
        # Обчислення помилки
        .....
        # Обчислення градієнта (похідна MSE за b)
        .....
        # Оновлення ваг
        .....
        # Збереження поточного MSE для історії
        .....
        .....
    return b, cost_history

```

5. Навчання моделі: Викличте `gradient_descent` на навчальних даних (`X_train_final`, `y_train`) .

```

b_final, cost_history = gradient_descent(X_train_final, y_train, b, learning_rate,
epochs)

```

6. Візуалізація навчання: Побудуйте графік `cost_history` по ітераціях.

Переконайтеся, що функція втрат зменшується. Якщо ні — потрібно зменшити `learning_rate` .

4. Оцінка якості моделі на тестових даних

1. Прогнозування: Для `X_test_final` зробіть прогноз: `y_pred = X_test_final.dot(b_final)` .

2. Розрахунок метрик:

```

* MSE: mse_test = np.mean((y_test - y_pred) ** 2)
* R2 Score: r2 = 1 - (np.sum((y_test - y_pred) ** 2) / np.sum((y_test -
np.mean(y_test)) ** 2))

```

3. Порівняння з бейзлайном: Порівняйте метрики з простою моделлю (наприклад, якщо б ми завжди передбачали середнє значення y_{train}).

5. Перевірка припущень класичної лінійної регресії (6 припущень)

Це критично важлива частина завдання для перевірки коректності побудованої моделі.

1. Лінійність зв'язку: Залишки не повинні мати систематичного паттерну.

Побудуйте графік залишків від передбачених значень ($residuals = y_{test} - y_{pred}$).

- * Очікування: Залишки повинні випадково розкидатися навколо нуля.

- * Проблема: Якщо видно кривий паттерн (наприклад, U-подібний), це означає, що модель не враховує якусь нелінійність.

2. Відсутність мультиколінеарності: Предиктори не повинні бути сильно корельовані між собою.

- * Метод: Розрахуйте VIF (Variance Inflation Factor) для кожної ознаки. $VIF = 1 / (1 - R^2_i)$, де R^2_i — це коефіцієнт детермінації для регресії i -ої ознаки на всі інші ознаки.

- * Очікування: $VIF < 5$ (або консервативніше < 10) вказує на відсутність серйозної мультиколінеарності.

3. Нормальність залишків: Залишки повинні мати нормальний розподіл.

- * Метод 1: Побудуйте Q-Q plot (Quantile-Quantile plot). Використайте `scipy.stats.probplot(residuals, plot=plt)` .

- * Метод 2: Виконайте статистичний тест (наприклад, Shapiro-Wilk test або Jarque-Bera test). $p\text{-value} > 0.05$ говорить про нормальність.

- * Очікування: Точки на Q-Q plot повинні лежати приблизно на прямій лінії.

4. Гомоскедастичність (постійна дисперсія залишків): Дисперсія залишків має бути постійною.

- * Метод: Той самий графік, що й для лінійності (залишки від прогнозів).

- * Очікування: "Хмара" точок повинна мати приблизно однакову ширину по всій довжині.

- * Проблема: Якщо хмара нагадує "воронку" (дисперсія зростає/зменшується), це гетероскедастичність.

5. Відсутність автокореляції залишків: Залишки не повинні бути корельовані між собою (важливо для тимчасових рядів).

- * Метод: Для даних, що не є часовими рядами, часто досить графіка. Для перевірки використовуйте тест Дарбіна-Вотсона

(statsmodels.stats.stattools.durbin_watson). Значення близьке до 2.0 означає відсутність автокореляції.

6. Відсутність викидів та впливових спостережень:

- * Викиди: Спостереження, що мають великі залишки. Шукайте точки, які далеко від нуля на графіку залишків.

- * Впливові спостереження: Спостереження, що сильно впливають на нахил лінії регресії. Для їх виявлення використовують Cook's distance . Спостереження з $\text{Cook's } D > 4/n$ (де n — кількість спостережень) вважаються впливовими.

6. Представлення результатів

Результати — код у Jupyter Notebook, який містить:

1. Код: Добре прокоментований код для всіх кроків.

2. Результати:

- * Знайдені коефіцієнти b_{final} .

- * Графік збіжності функції втрат.

- * Значення метрик MSE та R^2 на тестовій вибірці.

3. Аналіз припущень:

- * Графік залишків vs. прогнозів.

- * Q-Q plot залишків.

- * Результати розрахунку VIF для ознак.

- * Висновки щодо того, чи виконуються кожне з 6 припущень. Якщо ні — поясніть, що це може означати для надійності моделі.

4. Висновок: Загальний висновок про якість побудованої моделі та її придатність для прогнозування успішності студентів.

Критерії оцінювання:

- * *Коректність реалізації градієнтного спуску.*

- * *Якість попередньої обробки даних (обробка категоріальних змінних, масштабування).*

- * *Глибина та правильність аналізу припущень МНК.*

- * *Чистота, структурованість та коментарі коду.*

Допоміжні лінки:

1. [Linear Regression](#)

2. [YouTube](#)